

Projeto Prático de Inteligência Artificial:

*Inteligência Artificial aplicada à medicina - Detecção e
tumores cerebrais*

Aluno: Eduardo Octávio de Paula Souza

Professor: Ângelo Magno de Jesus

Ouro Branco, 26 de Junho de 2026

1 Introdução:

1.1 Link do Video: <https://youtu.be/Cxfi7JkjsDc>

1.2 Problema:

Tumores cerebrais são uma das doenças mais graves que existem, e identificar o tipo certo de tumor é essencial para que o médico escolha o tratamento correto. Hoje esse diagnóstico é feito por médicos especialistas que analisam imagens de ressonância magnética do cérebro. Este processo exige experiência, tempo e está sujeito a erros humanos.

A Inteligência Artificial pode ajudar nesse processo. Treinando um modelo com milhares de imagens já classificadas por médicos, é possível criar um sistema capaz de identificar automaticamente o tipo de tumor presente na imagem. Para o trabalho, foi utilizado um dataset com 7.200 imagens de ressonância magnética, divididas em quatro categorias:

- **Glioma:** tumor que surge nas células de suporte do cérebro
- **Meningioma:** tumor que surge nas membranas que envolvem o cérebro
- **Tumor Hipofisário:** tumor na glândula hipófise, localizada na base do cérebro
- **Sem tumor:** imagens de cérebros saudáveis

O objetivo é treinar modelos de aprendizado de máquina capazes de olhar para uma imagem de ressonância e classificá-la corretamente em uma dessas quatro categorias.

1.2 MLP - Multiplayer Perceptro:

MLP é um dos tipos mais simples de rede neural. Ela funciona recebendo os dados de entrada e vai passando por várias camadas de neurônios e chegando a uma resposta no final. Cada neurônio recebe valores, faz um cálculo simples e passa o resultado pra próxima camada, se repetindo até o final.

No caso das imagens, o MLP comprime a imagem inteira em uma lista longa de pixels e processa tudo de uma vez. O problema disso é que ela perde a noção de onde cada pixel estava na imagem, ela não entende que pixels próximos formam bordas ou regiões. Por isso, o MLP tende a ter desempenho mais fraco em tarefas com imagens.

1.3 Transformer

O Transformer é uma rede neural moderna, criada originalmente para processar texto, mas que hoje é amplamente usada também para imagens. A grande diferença em relação ao MLP é o mecanismo de atenção, em vez de processar tudo de uma vez de forma igual, o Transformer aprende a prestar mais atenção nas partes mais importantes da imagem.

Para funcionar com imagens, o Transformer divide a imagem em pequenos pedaços chamados de patches, processa cada pedaço e aprende as relações entre eles. Isso permite que ele entenda o contexto da imagem, por exemplo, que uma região escura no centro pode indicar a presença de um tumor. Por isso, o Transformer costuma ter desempenho muito superior a qualquer outro modelo em relação a classificar imagens.

2 Código Base:

Para provar que o Transformer é melhor que o MLP na classificação de tumores cerebrais, os dois modelos foram testados com os seguintes códigos como base. Cada experimento parte daqui e varia algum parâmetro específico.

2.1 Carregar das Imagens

```
def carregar_dados():|

    # Normaliza os pixels para 0 e 1
    datagen = ImageDataGenerator(rescale=1.0 / 255.0)

    train_data = datagen.flow_from_directory(
        TRAIN_DIR,
        target_size=IMG_SIZE,
        batch_size=BATCH_SIZE,
        class_mode="categorical",
        classes=CLASSES,
        seed=SEED,
        shuffle=True,
    )

    test_data = datagen.flow_from_directory(
        TEST_DIR,
        target_size=IMG_SIZE,
        batch_size=BATCH_SIZE,
        class_mode="categorical",
        classes=CLASSES,
        seed=SEED,
        shuffle=False,
    )

    print(f"\nClasses encontradas: {train_data.class_indices}")
    print(f"Batches de treino : {len(train_data)}")
    print(f"Batches de teste : {len(test_data)}")

    return train_data, test_data
```

2.2 Modelo MLP

```
def criar_mlp(neuronios=128, dropout=0.3, l2=0.0):
    reg = regularizers.L2(l2) if l2 > 0 else None

    modelo = models.Sequential([
        layers.Input(shape=INPUT_SHAPE),
        layers.Flatten(), # transforma a imagem em uma lista de numeros
        layers.Dense(neuronios, activation="relu", kernel_regularizer=reg),
        layers.Dropout(dropout),
        layers.Dense(neuronios // 2, activation="relu", kernel_regularizer=reg),
        layers.Dropout(dropout),
        layers.Dense(NUM_CLASSES, activation="softmax"), # probabilidade pra cada classe
    ], name="MLP")

    modelo.compile(optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"])
    return modelo
```

2.3 Modelo Transformers

```
def criar_transformer(patch_size=16, dim=64, num_heads=4, num_blocos=4, dropout=0.1):
    num_patches = (IMG_SIZE[0] // patch_size) * (IMG_SIZE[1] // patch_size)

    entradas = layers.Input(shape=INPUT_SHAPE)

    # divide a imagem em patches
    x = layers.Conv2D(dim, kernel_size=patch_size, strides=patch_size)(entradas)
    x = layers.Reshape((num_patches, dim))(x)

    # informa a posicao de cada patche
    posicoes = tf.range(start=0, limit=num_patches, delta=1)
    pos_embed = layers.Embedding(input_dim=num_patches, output_dim=dim)(posicoes)
    x = x + pos_embed

    # passa pelos blocos transformer onde acontece a atencao
    for _ in range(num_blocos):
        x = _bloco_transformer(x, num_heads, dim, dropout)

    # classifica com base no que aprendeu
    x = layers.LayerNormalization(epsilon=1e-6)(x)
    x = layers.GlobalAveragePooling1D()(x)
    x = layers.Dropout(dropout)(x)
    saidas = layers.Dense(NUM_CLASSES, activation="softmax")(x)

    modelo = models.Model(entradas, saidas, name="Transformer")
    modelo.compile(optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"])
    return modelo
```

```
def _bloco_transformer(x, num_heads, dim, dropout):
    norm1 = layers.LayerNormalization(epsilon=1e-6)(x)
    attn = layers.MultiHeadAttention(num_heads=num_heads, key_dim=dim, dropout=dropout)(norm1, norm1)
    x = layers.Add()([attn, x])

    norm2 = layers.LayerNormalization(epsilon=1e-6)(x)
    ff = layers.Dense(dim * 2, activation="relu")(norm2)
    ff = layers.Dropout(dropout)(ff)
    ff = layers.Dense(dim)(ff)
    x = layers.Add()([ff, x])
    return x
```

2.4 Cálculo das Métricas

```
def avaliar(modelo, test_data):

    test_data.reset()
    y_pred = np.argmax(modelo.predict(test_data, verbose=0), axis=1)
    y_true = test_data.classes

    return {
        # total de acertos
        "acuracia": accuracy_score(y_true, y_pred),
        # de tudo que o modelo disse ser tumor X, quantos realmente eram?
        "precisao": precision_score(y_true, y_pred, average="macro", zero_division=0),
        # de todos os tumores X que existiam, quantos o modelo encontrou?
        "recall": recall_score(y_true, y_pred, average="macro", zero_division=0),
        # media entre precisao e recall
        "f_score": f1_score(y_true, y_pred, average="macro", zero_division=0),
    }

def salvar_csv(experimento, tecnica, descricao, metricas):
    # cria o arquivo csv
    novo = not os.path.exists(CSV_RESULTADOS)

    with open(CSV_RESULTADOS, "a", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=COLUMNAS)
        if novo:
            writer.writeheader()
        writer.writerow({
            "experimento": experimento,
            "tecnica": tecnica,
            "descricao": descricao,
            "acuracia": round(metricas["acuracia"], 4),
            "precisao": round(metricas["precisao"], 4),
            "recall": round(metricas["recall"], 4),
            "f_score": round(metricas["f_score"], 4),
        })

    print(f"Resultado de '{experimento}' salvo em '{CSV_RESULTADOS}'")
```

2.5 Código de experimentos

```
EXPERIMENTOS = [
    ("Exp1", "MLP", "base (neuronios=128, dropout=0.3)",
     lambda: criar_mlp(neuronios=128, dropout=0.3)),
    ("Exp2", "MLP", "mais neuronios=256",
     lambda: criar_mlp(neuronios=256, dropout=0.3)),
    ("Exp3", "MLP", "dropout alto=0.5 (anti-overfit)",
     lambda: criar_mlp(neuronios=128, dropout=0.5)),
    ("Exp4", "MLP", "regularizacao L2=0.001 (anti-overfit)",
     lambda: criar_mlp(neuronios=128, dropout=0.3, l2=0.001)),
    ("Exp5", "Transformer", "base (dim=64, heads=4, blocos=4, dropout=0.1)",
     lambda: criar_transformer(dim=64, num_heads=4, num_blocos=4, dropout=0.1)),
    ("Exp6", "Transformer", "maior dim=128",
     lambda: criar_transformer(dim=128, num_heads=4, num_blocos=4, dropout=0.1)),
    ("Exp7", "Transformer", "mais heads=8",
     lambda: criar_transformer(dim=64, num_heads=8, num_blocos=4, dropout=0.1)),
    ("Exp8", "Transformer", "dropout alto=0.3 (anti-overfit)",
     lambda: criar_transformer(dim=64, num_heads=4, num_blocos=4, dropout=0.3)),
]

def rodar_experimentos():
    verificar_estrutura()
    train_data, test_data = carregar_dados()

    for nome, tecnica, descricao, criar_modelo in EXPERIMENTOS:
        print(f"\n{'='*60}")
        print(f" {nome} | {tecnica} | {descricao}")
        print(f"{'='*60}")

        modelo = criar_modelo()
        modelo.fit(train_data, epochs=EPOCAS, verbose=1)
        avaliar_e_salvar(nome, tecnica, descricao, modelo, test_data)

    print("\nTodos os experimentos concluídos! Veja 'resultados.csv'.")

if __name__ == "__main__":
    rodar_experimentos()
```

3 Experimentos

Com os modelos prontos, foram realizados 8 experimentos ao total, foram 4 com MLP e 4 com Transformer. Em cada experimento, um parâmetro diferente foi alterado para observar como isso afeta o desempenho do modelo. Essa variação é importante porque não existe uma configuração perfeita para todos os casos, logo é testando que se descobre o que funciona melhor para um problema específico.

1. **EXP 1 - MLP Base** - `criar_mlp(neuronios=128, dropout=0.3)`: Ponto de partida do MLP. Serve para ter um resultado de referência, sem nenhuma modificação, para comparar com os experimentos seguintes.

2. **EXP 2 - MLP com mais neurônios** - `criar_mlp(neuronios=256, dropout=0.3)`:
Aqui foi dobrado o número de neurônios (de 128 para 256) para dar ao modelo mais capacidade de aprender padrões. Mais neurônios significa mais memória para o modelo, mas também mais risco de decorar as imagens de treino.
3. **EXP3 - MLP com dropout alto** - `criar_mlp(neuronios=128, dropout=0.5)`:
O dropout foi aumentado de 30% para 50%. Isso significa que metade dos neurônios é desligada aleatoriamente durante o treino, uma técnica para evitar que o modelo decore as imagens em vez de aprender de verdade.
4. **EXP4 - MLP com regularização L2** - `criar_mlp(neuronios=128, dropout=0.3, l2=0.001)`: Foi adicionada a regularização L2, que penaliza o modelo quando ele coloca peso demais em conexões específicas. É outra técnica anti-overfitting, diferente do dropout, uma vez que ao desligar neurônios, ela limita o quanto cada neurônio pode influenciar o resultado.
5. **EXP5 - Transformer Base** - `criar_transformer(dim=64, num_heads=4, num_blocos=4, dropout=0.1)`: Ponto de partida do Transformer. Assim como o Exp1 para o MLP, serve de referência para medir o impacto das variações seguintes.
6. **EXP6 - Transformer com dimensão maior** - `criar_transformer(dim=128, num_heads=4, num_blocos=4, dropout=0.1)`: Testamos se aumentar a capacidade interna do Transformer melhora os resultados. Com dimensão 128, cada pedaço da imagem é representado com mais detalhes.
7. **EXP7 - Transformer com mais cabeças de atenção** - `criar_transformer(dim=64, num_heads=8, num_blocos=4, dropout=0.1)`: Testamos se o modelo se sai melhor prestando atenção em mais partes da imagem ao mesmo tempo, com 8 cabeças, ele analisa a imagem sob mais perspectivas simultaneamente.
8. **EXP8 - Transformer com dropout alto** - `criar_transformer(dim=64, num_heads=4, num_blocos=4, dropout=0.3)`: Testamos se o Transformer também se beneficia de um dropout mais agressivo, assim como foi testado no MLP no Exp3

4 Resultados

Após rodar os 8 experimentos com 20 épocas (cada modelo passou 20 vezes por todo o dataset dos tumores) os resultados foram salvos automaticamente em um arquivo CSV. A tabela abaixo mostra as 4 métricas avaliadas para cada experimento:

- **Acurácia:** de todas as imagens testadas, quantas o modelo classificou corretamente
- **Precisão:** das imagens que o modelo disse ser de determinado tumor, quantas realmente eram
- **Recall:** de todos os tumores que existiam no teste, quantos o modelo conseguiu encontrar
- **F-Score:** média entre precisão e recall, usada para ter uma visão geral do desempenho

Quanto mais próximo de 1,0 (ou 100%), melhor o desempenho do modelo.

Experimento	Técnica	Descrição	Acuracia	Precisao	Recall	F Score
Exp1	MLP	base (neurônios=128, dropout=0.3)	25%	6,25%	25%	10%
Exp2	MLP	mais neurônios=256	64,90%	60,40%	64,90%	58%
Exp3	MLP	dropout alto=0.5 (anti-overfit)	25%	6,25%	25%	10%
Exp4	MLP	regularizacao L2=0.001 (anti-overfit)	25%	6,25%	25%	10%
Exp5	Transformer	base (dim=64, heads=4, blocos=4, dropout=0.1)	81,10%	81,20%	81,10%	80,50%
Exp6	Transformer	maior dim=128	79,10%	80,80%	79,10%	78,40%
Exp7	Transformer	mais heads=8	82,10%	83,20%	82,10%	81,40%
Exp8	Transformer	dropout alto=0.3 (anti-overfit)	75,20%	76,90%	75,20%	74,90%

5 Resultados

O Transformer se mostrou claramente superior ao MLP neste experimento, sendo o melhor resultado o experimento 7, onde o resultado atingido foi de 82,1% de acurácia e 81,4% de F-Score. Por outro lado, o MLP teve desempenho bem abaixo, na qual três dos quatro experimentos ficaram travados em 25% de acurácia, que é exatamente a chance de acertar chutando entre 4 classes, ou seja, esses modelos não aprenderam nada útil. Só o Exp2, com 256 neurônios, conseguiu aprender algo, chegando a 64,9%.

Isso confirma o que foi explicado, o MLP não foi feito para imagens. Ao achatar a imagem em uma lista de números, ele perde toda a informação espacial que é justamente o que diferencia um tipo de tumor do outro em uma ressonância magnética. O Transformer, por sua vez, divide a imagem em pedacinhos e aprende as relações entre eles, prestando atenção nas partes mais importantes. O experimento 7 foi o melhor porque com 8 cabeças de atenção o modelo analisou a imagem sob mais perspectivas ao mesmo tempo, capturando mais detalhes relevantes.

Com mais épocas de treino, mais dados e ajustes nos parâmetros, é provável que o Transformer alcançasse resultados ainda melhores. Uma conjectura é que o experimento 6, com dimensão maior (128), teve desempenho levemente inferior ao base, possivelmente porque uma dimensão maior exige mais dados e épocas para aprender corretamente. Já o experimento 8, com dropout alto, teve o pior resultado entre os Transformers, sugerindo que o Transformer já controla bem o overfitting por conta própria e um dropout muito alto acaba atrapalhando o aprendizado. Tudo isso confirma que o Transformer é a escolha certa para classificação de imagens.